



Modelos - Aula 3

🕒 Criado em	7 de setembro de 2026 09:51
🏷️ Tags	

Antes de falar dos modelos, precisamos entender o que significa **modelar um sistema distribuído**.

Um sistema distribuído pode ser enorme e bastante complexo. Então, em vez de tentar analisar tudo ao mesmo tempo, usamos **modelos** para representar determinados aspectos do sistema.

De forma simples:

Um modelo é uma forma de simplificar e representar um sistema para podermos analisá-lo, projetá-lo e entender seu comportamento.

Existem duas categorias importantes nesta unidade:

- **Modelos arquiteturais** → preocupam-se com **como os componentes do sistema são organizados**.
- **Modelos fundamentais** → preocupam-se com **as características e problemas que afetam o funcionamento desses componentes**.

1. Modelos Arquiteturais

O modelo arquitetural responde principalmente:

"Quais são os componentes do sistema e como eles estão organizados e se comunicam?"

Ou seja, estamos olhando para a **estrutura do sistema**.

Podemos pensar em:

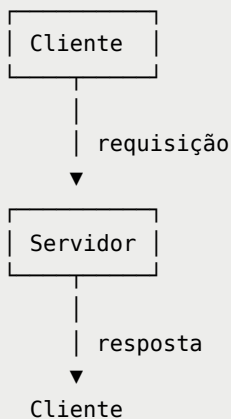
```
Componentes
  ↓
Como estão organizados?
  ↓
Como se comunicam?
  ↓
Quem depende de quem?
```

1.1 Cliente-Servidor

É uma das arquiteturas mais conhecidas.

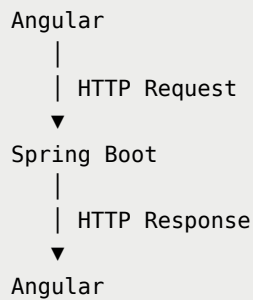
Temos dois papéis principais:

- **cliente** → solicita um serviço;
- **servidor** → fornece o serviço.



Exemplo

Quando seu navegador acessa uma API:



O Angular assume o papel de cliente e o Spring Boot fornece o serviço.

No mercado

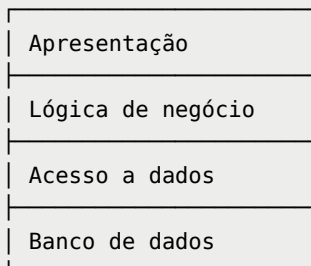
Essa arquitetura aparece o tempo todo:

- aplicações web;
- APIs REST;
- bancos de dados;
- serviços de autenticação;
- sistemas corporativos.

2. Arquitetura em camadas

Podemos organizar um sistema em diferentes camadas, onde cada uma possui uma responsabilidade.

Um exemplo clássico:



No seu contexto, poderia ser:

```
Angular
↓
Controller
↓
Service
↓
Repository
↓
MySQL
```

A vantagem é separar responsabilidades.

Por exemplo:

- Angular → interface;
- Controller → entrada das requisições;
- Service → regras de negócio;
- Repository → acesso aos dados;
- MySQL → armazenamento.

⚠ Uma observação importante

Arquitetura em camadas não significa necessariamente sistema distribuído.

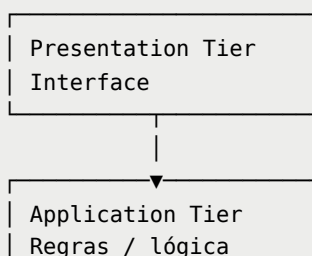
Podemos ter todas essas camadas rodando na mesma máquina.

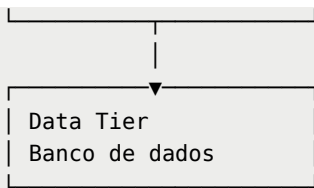
O conceito de distribuição depende de **onde os componentes estão executando e como se comunicam**, não simplesmente da quantidade de camadas.

3. Arquitetura de três camadas

Uma arquitetura bastante conhecida é a **three-tier architecture**.

Normalmente temos:





Por exemplo:

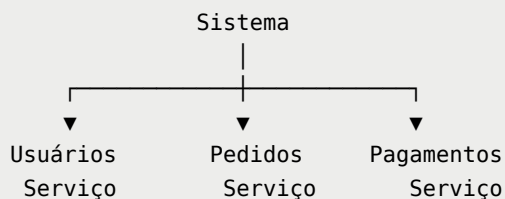
```
Angular
  ↓
Spring Boot
  ↓
MySQL
```

Isso é extremamente próximo do que você encontra no desenvolvimento web.

4. Arquitetura baseada em serviços

Em sistemas maiores, podemos separar funcionalidades em serviços diferentes.

Por exemplo:



Cada serviço pode ter responsabilidade própria.

Por exemplo:

```
Pedido
├── consulta usuário
├── calcula pedido
└── solicita pagamento
```

Essa ideia está muito relacionada ao conceito de **microserviços**.

Mas cuidado:

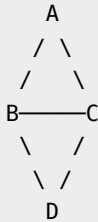
| **Sistema distribuído ≠ microserviços.**

Microserviços são **uma forma de arquitetar um sistema**, e geralmente resultam em um sistema distribuído, mas existem muitos sistemas distribuídos que não utilizam microserviços.

5. Arquitetura Peer-to-Peer (P2P)

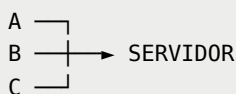
No modelo **peer-to-peer**, os participantes possuem papéis mais semelhantes.

Não existe necessariamente um servidor central responsável por tudo.

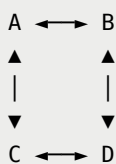


Cada participante, chamado de **peer**, pode tanto solicitar quanto fornecer recursos.

Isso é diferente do modelo tradicional:



No P2P:



Onde podemos encontrar essa ideia?

- compartilhamento distribuído;

- algumas redes de arquivos;
 - determinadas arquiteturas blockchain;
 - sistemas descentralizados.
-

6. Por que estudar modelos arquiteturais?

Porque quando você recebe um problema real, precisa tomar decisões como:

- | "Quem será responsável por essa função?"
- | "Esse componente deve ficar onde?"
- | "Quem conversa com quem?"
- | "Precisamos de um servidor central?"
- | "Podemos separar essa funcionalidade em outro serviço?"
- | "Como os componentes vão se comunicar?"

Essas são perguntas de **arquitetura de sistemas**.

7. Modelos Fundamentais

Agora muda um pouco a perspectiva.

Enquanto o modelo arquitetural olha principalmente para:

- | **"Como o sistema está organizado?"**

o modelo fundamental olha para:

- | **"Quais características do ambiente podem afetar o funcionamento do sistema?"**

Ele nos ajuda a raciocinar sobre coisas como:

- comunicação;
- tempo;
- falhas;

- concorrência;
- segurança.

Uma forma simples de lembrar:

```

MODELO ARQUITETURAL
  ↓
Estrutura
  ↓
"Como organizamos?"

MODELO FUNDAMENTAL
  ↓
Comportamento / problemas
  ↓
"O que pode acontecer?"

```

8. Modelo de interação

Um dos aspectos fundamentais é a **interação entre os processos**.

Precisamos considerar:

- quando os processos executam;
- quanto tempo uma mensagem pode levar;
- quanto tempo um processamento pode demorar;
- se existe ou não sincronização entre os processos.

Imagine:

```

Processo A
  |
  | mensagem
  ▼
REDE
  |
  | ???
  ▼
Processo B

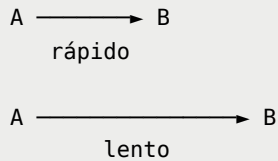
```

A pergunta é:

Quanto tempo essa mensagem vai levar?

Em uma rede real, não existe garantia de que ela chegará exatamente quando esperamos.

Pode haver:



Essa incerteza é fundamental para projetar sistemas distribuídos.

9. Modelo síncrono e assíncrono

Uma forma de analisar sistemas distribuídos é considerar o comportamento do tempo.

Sistema síncrono

Existem limites conhecidos para determinadas operações.

Por exemplo:

```
Mensagem A → B  
  
Tempo máximo esperado: 100 ms
```

Isso permite fazer algumas suposições sobre o comportamento do sistema.

Sistema assíncrono

Não podemos assumir limites confiáveis para o tempo de comunicação ou processamento.

```
Mensagem A → B  
  
Pode chegar:  
10 ms  
100 ms  
1 s  
...
```

E isso gera uma dificuldade enorme.

Se B não respondeu, A não sabe necessariamente se:

B caiu

OU:

B está apenas demorando

Esse conceito será importante mais adiante.

10. Modelo de falhas

Outro modelo fundamental descreve **como os componentes podem falhar**.

Um sistema distribuído pode apresentar diferentes tipos de falha.

Por exemplo:

Falha por parada

Um processo simplesmente deixa de funcionar.

A ✓
B ✗
C ✓

Falha de comunicação

Uma mensagem pode:

- não chegar;
- chegar atrasada;
- ter sua resposta perdida.

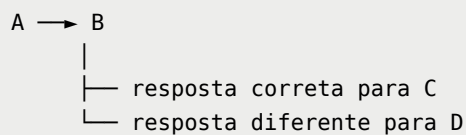
A —X→ B
mensagem perdida

Falha arbitrária

Também chamada de **falha bizantina**.

É uma situação mais complicada na qual um componente pode apresentar um comportamento incorreto ou inconsistente.

Por exemplo:

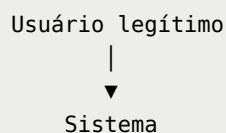


Esse tipo de problema é especialmente relevante em sistemas que precisam funcionar mesmo quando alguns participantes não são confiáveis.

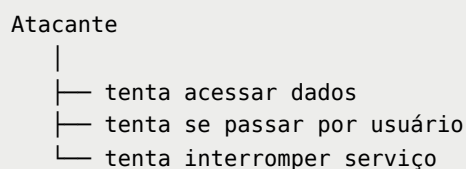
11. Modelo de segurança

Também podemos modelar as possíveis ameaças ao sistema.

Por exemplo:



Mas existe:



Precisamos pensar em:

- quem é confiável;
- quem pode acessar determinado recurso;
- como autenticar;

- como proteger mensagens;
 - como detectar comportamentos maliciosos.
-

12. A diferença que você precisa guardar

Essa é provavelmente a parte mais importante desse material.

Modelo arquitetural

Olha para a **estrutura**.

| Como os componentes estão organizados?

Exemplo:

```
Cliente → Servidor
```

OU:

```
Angular → Backend → Banco
```

OU:

```
Serviço A ↔ Serviço B ↔ Serviço C
```

Modelo fundamental

Olha para as **propriedades e problemas do funcionamento**.

| Como os componentes e o ambiente podem se comportar?

Por exemplo:

```
Comunicação  
↓  
Tempo  
↓  
Falhas
```

↓
Segurança

Tabela para memorizar

Modelo	O que analisa?	Pergunta
Arquitetural	Estrutura do sistema	Como organizamos os componentes?
Interação	Comunicação e tempo	Como os processos interagem?
Falhas	Comportamentos incorretos	O que acontece quando algo falha?
Segurança	Ameaças e proteção	O que pode ser atacado e como protegemos?

Ligação com o mercado

Essa parte da matéria começa a ficar **bem relevante para arquitetura de software**.

Quando você estiver trabalhando em um projeto e alguém disser:

| "Precisamos transformar essa aplicação em uma arquitetura distribuída."

Você não deveria pensar apenas:

| "Vamos criar vários servidores."

Você deveria começar a perguntar:

1. Quais componentes teremos?
↓
2. Como eles serão organizados?
↓
3. Como irão se comunicar?
↓
4. O que acontece se um deles falhar?
↓
5. Como vamos escalar?
↓
6. Como vamos proteger a comunicação?
↓

7. O que acontece quando vários componentes trabalham simultaneamente?

Essa forma de pensar é uma das coisas mais valiosas que você pode tirar dessa disciplina para o mercado.

E tem uma conexão muito boa com o que você já está estudando: quando você olha para **Angular + Spring Boot + MySQL**, depois começa a adicionar **Redis, mensageria, microsserviços, Docker e cloud**, você está entrando cada vez mais no território de sistemas distribuídos.